

Questions 61-80 - Full Solutions

Arrays (Medium/Hard) + Binary Search start - In Depth - C++ & Python

What's inside. Items 61-80 - the toughest array problems on the sheet, plus your first binary search. This block is heavy on the classics interviewers love: **Next Permutation, spiral/rotate matrix, 3-Sum, 4-Sum, Count Inversions, Reverse Pairs, Maximum Product Subarray**, and the prefix-sum family (subarrays with sum/XOR K). Each solved in **both C++ and Python** with approach, code, and complexity. All code compiled with `g++ -std=c++17` and run against test cases in Python.

These are harder - read the Approach slowly, and don't be surprised if a few need a second pass. The merge-sort-based counting (inversions, reverse pairs) and the two-pointer-after-sort pattern (3-Sum, 4-Sum) are the big new techniques here.

Arrays - Medium (Q61-Q67)

Q61. Next Permutation

Medium

■ **Approach:** Find the next lexicographic arrangement. 1) Scan from the right for the first index i where $a[i] < a[i+1]$ (the 'pivot'). 2) Find the rightmost element bigger than $a[i]$ and swap them. 3) Reverse the suffix after i (turning it from descending to ascending = smallest). If no pivot exists, the array is the last permutation - just reverse it.

C++

```
void nextPerm(vector<int>& a){
    int n = a.size(), i = n-2;
    while(i >= 0 && a[i] >= a[i+1]) i--; // find pivot
    if(i >= 0){
        int j = n-1;
        while(a[j] <= a[i]) j--; // next bigger
        swap(a[i], a[j]);
    }
    reverse(a.begin()+i+1, a.end()); // smallest suffix
}
```

PYTHON

```
def next_permutation(a):
    n = len(a); i = n-2
    while i >= 0 and a[i] >= a[i+1]: i -= 1
    if i >= 0:
        j = n-1
        while a[j] <= a[i]: j -= 1
        a[i], a[j] = a[j], a[i]
    a[i+1:] = a[i+1:][::-1]
    return a
```

Complexity & notes: $O(n)$, $O(1)$. C++ also has `std::next_permutation` - but interviewers want the manual 3-step logic.

Q62. Leaders in an Array

Easy

■ **Approach:** A leader is greater than everything to its right. Scan from the RIGHT keeping a running max; an element is a leader if it beats the max-so-far. Reverse the collected list at the end to restore order.

C++

```
vector<int> leaders(vector<int>& a){
    vector<int> r; int mx = INT_MIN;
    for(int i = a.size()-1; i >= 0; i--)
        if(a[i] > mx){ r.push_back(a[i]); mx = a[i]; }
    reverse(r.begin(), r.end());
    return r;
}
```

PYTHON

```
def leaders(a):
    res = []; mx = float("-inf")
    for x in reversed(a):
        if x > mx: res.append(x); mx = x
    return res[::-1]
```

Complexity & notes: $O(n)$, $O(1)$ extra. Scanning from the right turns an $O(n^2)$ 'check all to the right' into $O(n)$.

Q63. Longest Consecutive Sequence

Medium

■ **Approach:** Put all numbers in a hash set. A number x STARTS a run only if $x-1$ is absent. From each such start, walk $x, x+1, x+2...$ counting the run length. Each number is visited at most twice, so it's $O(n)$ - not $O(n \log n)$ sorting.

C++

```
int longestConsec(vector<int>& a){
    unordered_set<int> s(a.begin(), a.end());
    int best = 0;
    for(int x : s){
        if(!s.count(x-1)){           // start of a run
            int y = x;
            while(s.count(y+1)) y++;
            best = max(best, y - x + 1);
        }
    }
    return best;
}
```

PYTHON

```
def longest_consecutive(a):
    s = set(a); best = 0
    for x in s:
        if x-1 not in s:           # start of a run
            y = x
            while y+1 in s: y += 1
            best = max(best, y - x + 1)
    return best
```

Complexity & notes: $O(n)$ time, $O(n)$ space. The 'only start counting if $x-1$ is missing' check is what keeps it linear.

Q64. Set Matrix Zeros

Medium

■ **Approach:** If a cell is 0, its whole row and column become 0. To do it in $O(1)$ space, use the matrix's own first row and column as markers: mark $m[i][0]$ and $m[0][j]$ when $m[i][j]==0$. Handle the first row/col separately with two flags (they'd otherwise mark themselves).

C++

```
void setZeros(vector<vector<int>>& m){
    int R=m.size(), C=m[0].size(); bool r0=false, c0=false;
    for(int j=0;j<C;j++) if(m[0][j]==0) r0=true;
    for(int i=0;i<R;i++) if(m[i][0]==0) c0=true;
    for(int i=1;i<R;i++) for(int j=1;j<C;j++)
        if(m[i][j]==0){ m[i][0]=0; m[0][j]=0; } // mark
    for(int i=1;i<R;i++) for(int j=1;j<C;j++)
        if(m[i][0]==0 || m[0][j]==0) m[i][j]=0; // apply
    if(r0) for(int j=0;j<C;j++) m[0][j]=0;
    if(c0) for(int i=0;i<R;i++) m[i][0]=0;
}
```

PYTHON

```
def set_zeros(m):
    R,C = len(m), len(m[0])
    r0 = any(m[0][j]==0 for j in range(C))
    c0 = any(m[i][0]==0 for i in range(R))
    for i in range(1,R):
        for j in range(1,C):
            if m[i][j]==0: m[i][0]=0; m[0][j]=0 # mark
    for i in range(1,R):
        for j in range(1,C):
            if m[i][0]==0 or m[0][j]==0: m[i][j]=0 # apply
    if r0:
        for j in range(C): m[0][j]=0
    if c0:
        for i in range(R): m[i][0]=0
    return m
```

Complexity & notes: $O(R*C)$ time, $O(1)$ space. Naive uses $O(R+C)$ marker arrays; this reuses row 0 / col 0 to reach $O(1)$.

Q65. Rotate Matrix by 90 Degrees

Medium

■ **Approach:** Two steps: TRANSPOSE (swap $m[i][j]$ with $m[j][i]$) then REVERSE each row. Transpose flips over the diagonal; reversing rows completes the clockwise rotation. In place, no extra matrix.

C++

```
void rotate90(vector<vector<int>>& m){
    int n = m.size();
    for(int i=0;i<n;i++)
        for(int j=i+1;j<n;j++)
            swap(m[i][j], m[j][i]);    // transpose
    for(auto& row : m)
        reverse(row.begin(), row.end()); // reverse rows
}
```

PYTHON

```
def rotate90(m):
    n = len(m)
    for i in range(n):
        for j in range(i+1, n):
            m[i][j], m[j][i] = m[j][i], m[i][j] # transpose
    for row in m:
        row.reverse() # reverse rows
    return m
```

Complexity & notes: $O(n^2)$ time, $O(1)$ space. Transpose-then-reverse is the cleanest rotate. (Reverse columns instead for anti-clockwise.)

Q66. Print Matrix in Spiral Order

Medium

■ **Approach:** Keep four boundaries: top, bottom, left, right. Print the top row (L->R), right column (T->B), bottom row (R->L), left column (B->T), shrinking the boundary after each edge. The two inner if-checks prevent re-printing a single remaining row/column.

C++

```
vector<int> spiral(vector<vector<int>>& m){
    vector<int> r; if(m.empty()) return r;
    int top=0, bot=m.size()-1, left=0, right=m[0].size()-1;
    while(top<=bot && left<=right){
        for(int j=left;j<=right;j++) r.push_back(m[top][j]); top++;
        for(int i=top;i<=bot;i++) r.push_back(m[i][right]); right--;
        if(top<=bot){ for(int j=right;j>=left;j--) r.push_back(m[bot][j]); bot--; }
        if(left<=right){ for(int i=bot;i>=top;i--) r.push_back(m[i][left]); left++; }
    }
    return r;
}
```

PYTHON

```
def spiral(m):
    if not m: return []
    res=[]; top=0; bot=len(m)-1; left=0; right=len(m[0])-1
    while top<=bot and left<=right:
        for j in range(left,right+1): res.append(m[top][j])
        top+=1
        for i in range(top,bot+1): res.append(m[i][right])
        right-=1
        if top<=bot:
            for j in range(right,left-1,-1): res.append(m[bot][j]);
            bot-=1
        if left<=right:
            for i in range(bot,top-1,-1): res.append(m[i][left]);
            left+=1
    return res
```

Complexity & notes: $O(R*C)$ time, $O(1)$ extra. The four-boundary walk is the standard matrix-traversal template.

Q67. Count Subarrays with Sum K

Medium

■ **Approach:** Prefix sums + hash map of COUNTS. For running sum s , the number of subarrays ending here that sum to K equals how many earlier prefixes equalled $s-K$. Seed the map with $\{0:1\}$ for subarrays starting at index 0.

C++

```
int countSumK(vector<int>& a, int k){
    unordered_map<long,int> pre; pre[0]=1;
    long s=0; int cnt=0;
    for(int x : a){
        s += x;
        cnt += pre[s-k];    // subarrays ending here
        pre[s]++;
    }
    return cnt;
}
```

PYTHON

```
from collections import defaultdict
def count_sum_k(a, k):
    pre = defaultdict(int); pre[0]=1
    s = 0; cnt = 0
    for x in a:
        s += x
        cnt += pre[s-k]    # subarrays ending here
        pre[s] += 1
    return cnt
```

Complexity & notes: $O(n)$ time, $O(n)$ space. Same prefix-sum idea as Q53, but COUNTING subarrays (store counts, not first index).

Arrays - Hard (Q68-Q79)

Q68. Pascal's Triangle

Medium

■ **Approach:** Each row starts and ends with 1; every inner value is the sum of the two values above it ($\text{prev}[j-1] + \text{prev}[j]$). Build row by row from the previous one.

C++

```
vector<vector<int>> pascal(int n){
    vector<vector<int>> res = {{1}};
    for(int i=1;i<n;i++){
        vector<int>& p = res.back();
        vector<int> row = {1};
        for(int j=1;j<i;j++) row.push_back(p[j-1]+p[j]);
        row.push_back(1);
        res.push_back(row);
    }
    return res;
}
```

PYTHON

```
def pascal(n):
    res = [[1]]
    for i in range(1, n):
        prev = res[-1]; row = [1]
        for j in range(1, i): row.append(prev[j-1]+prev[j])
        row.append(1); res.append(row)
    return res
```

Complexity & notes: $O(n^2)$ time & space. A single element r,c is $C(r-1, c-1)$; a full row uses the nCr running-product trick.

Q69. Majority Element (> n/3)

Medium

■ **Approach:** At most TWO elements can appear more than $n/3$ times. Extended Boyer-Moore with two candidates and two counters. Then VERIFY both with a second pass (unlike $>n/2$, a $>n/3$ candidate isn't guaranteed).

C++

```
vector<int> majN3(vector<int>& a){
    int c1=INT_MIN,c2=INT_MIN,n1=0,n2=0;
    for(int x : a){
        if(x==c1) n1++;
        else if(x==c2) n2++;
        else if(n1==0){ c1=x; n1=1; }
        else if(n2==0){ c2=x; n2=1; }
        else { n1--; n2--; }
    }
    vector<int> r;
    for(int c : {c1,c2})
        if(count(a.begin(),a.end(),c) > (int)a.size()/3) r.push_back(c);
    return r;    // (dedupe if c1==c2)
}
```

PYTHON

```
def maj_n3(a):
    c1=c2=None; n1=n2=0
    for x in a:
        if x==c1: n1+=1
        elif x==c2: n2+=1
        elif n1==0: c1=x; n1=1
        elif n2==0: c2=x; n2=1
        else: n1-=1; n2-=1
    return sorted({c for c in (c1,c2)
                     if c is not None and a.count(c) > len(a)//3})
```

Complexity & notes: $O(n)$ time, $O(1)$ space. Two-candidate voting + a verification pass. The verify step is essential for $>n/3$.

Q70. 3-Sum

Medium

■ **Approach:** Sort first. Fix each i , then two-pointer (l , r) on the rest to find pairs summing to $-a[i]$. Skip duplicate i , l , r values to keep triplets unique. Sorting is what enables both the two pointers and the dedup.

C++

```
vector<vector<int>> threeSum(vector<int> a){
    sort(a.begin(), a.end());
    vector<vector<int>> res; int n=a.size();
    for(int i=0;i<n-2;i++){
        if(i>0 && a[i]==a[i-1]) continue;
        int l=i+1, r=n-1;
        while(l<r){
            int s=a[i]+a[l]+a[r];
            if(s==0){ res.push_back({a[i],a[l],a[r]}); l++; r--;
                while(l<r&& a[l]==a[l-1]) l++;
                while(l<r&& a[r]==a[r+1]) r--; }
            else if(s<0) l++; else r--;
        }
    }
    return res;
}
```

PYTHON

```
def three_sum(a):
    a=sorted(a); res=[]; n=len(a)
    for i in range(n-2):
        if i>0 and a[i]==a[i-1]: continue
        l,r=i+1,n-1
        while l<r:
            s=a[i]+a[l]+a[r]
            if s==0:
                res.append([a[i],a[l],a[r]]); l+=1; r-=1
                while l<r and a[l]==a[l-1]: l+=1
                while l<r and a[r]==a[r+1]: r-=1
            elif s<0: l+=1
            else: r-=1
    return res
```

Complexity & notes: $O(n^2)$ time, $O(1)$ extra (besides output). 'Sort + fix one + two pointers' - a top-tier interview pattern.

Q71. 4-Sum

Hard

■ **Approach:** Extend 3-Sum by one more fixed loop: fix i and j , then two-pointer on the rest for the remaining two numbers summing to target - $a[i]$ - $a[j]$. Skip duplicates at all four levels. Use a 64-bit sum to avoid overflow.

C++

```
vector<vector<int>> fourSum(vector<int> a, int t){
    sort(a.begin(), a.end());
    vector<vector<int>> res; int n=a.size();
    for(int i=0;i<n-3;i++){
        if(i>0&&a[i]==a[i-1]) continue;
        for(int j=i+1;j<n-2;j++){
            if(j>i+1&&a[j]==a[j-1]) continue;
            int l=j+1, r=n-1;
            while(l<r){
                long s=(long)a[i]+a[j]+a[l]+a[r];
                if(s==t){ res.push_back({a[i],a[j],a[l],a[r]}); l++; r--;
                    while(l<r&&a[l]==a[l-1]) l++;
                    while(l<r&&a[r]==a[r+1]) r--; }
                else if(s<t) l++; else r--;
            }
        }
    }
    return res;
}
```

PYTHON

```
def four_sum(a, t):
    a=sorted(a); res=[]; n=len(a)
    for i in range(n-3):
        if i>0 and a[i]==a[i-1]: continue
        for j in range(i+1,n-2):
            if j>i+1 and a[j]==a[j-1]: continue
            l,r=j+1,n-1
            while l<r:
                s=a[i]+a[j]+a[l]+a[r]
                if s==t:
                    res.append([a[i],a[j],a[l],a[r]]); l+=1; r-=1
                    while l<r and a[l]==a[l-1]: l+=1
                    while l<r and a[r]==a[r+1]: r-=1
                elif s<t: l+=1
                else: r-=1
    return res
```

Complexity & notes: $O(n^3)$ time. Same reduction idea as 3-Sum with one extra fixed loop. Watch overflow - use long in C++.

Q72. Largest Subarray with Sum 0

Medium

■ **Approach:** Prefix sum + hash map of FIRST index. If the same prefix sum appears twice (at i and j), the subarray between them sums to 0; its length is $i - j$. Keep the earliest index of each prefix to maximize length. If prefix itself is 0, the whole prefix qualifies.

C++

```
int largestZeroSum(vector<int>& a){
    unordered_map<long,int> pre; long s=0; int best=0;
    for(int i=0;i<(int)a.size();i++){
        s += a[i];
        if(s==0) best = i+1;
        else if(pre.count(s)) best = max(best, i - pre[s]);
        else pre[s] = i;          // keep earliest
    }
    return best;
}
```

PYTHON

```
def largest_zero_sum(a):
    pre = {}; s = 0; best = 0
    for i, x in enumerate(a):
        s += x
        if s == 0: best = i+1
        elif s in pre: best = max(best, i - pre[s])
        else: pre[s] = i          # keep earliest
    return best
```

Complexity & notes: $O(n)$ time, $O(n)$ space. Store the FIRST index (not a count) - here we want the longest, not how many.

Q73. Count Subarrays with XOR K

Hard

■ **Approach:** Prefix XOR + hash map of counts. If the running XOR is x , a subarray ending here XORs to K when some earlier prefix equalled $x \oplus K$ (because $x \oplus (x \oplus K) = K$). Count those. Seed $\{0:1\}$.

C++

```
int countXorK(vector<int>& a, int k){
    unordered_map<int,int> pre; pre[0]=1;
    int x=0, cnt=0;
    for(int v : a){
        x ^= v;
        cnt += pre[x ^ k];    // earlier prefixes
        pre[x]++;
    }
    return cnt;
}
```

PYTHON

```
from collections import defaultdict
def count_xor_k(a, k):
    pre = defaultdict(int); pre[0]=1
    x = 0; cnt = 0
    for v in a:
        x ^= v
        cnt += pre[x ^ k]    # earlier prefixes
        pre[x] += 1
    return cnt
```

Complexity & notes: $O(n)$ time, $O(n)$ space. The XOR analogue of 'count subarrays with sum K ' - same structure, $\oplus$ instead of $-$.

Q74. Merge Overlapping Intervals

Medium

■ **Approach:** Sort by start. Walk through; if the current interval starts within the last merged one ($\text{start} \leq \text{last.end}$), extend last.end to the max. Otherwise start a new interval. Sorting guarantees overlaps are adjacent.

C++

```
vector<vector<int>> mergeIntervals(vector<vector<int>> iv){
    sort(iv.begin(), iv.end());
    vector<vector<int>> res;
    for(auto& in : iv){
        if(!res.empty() && in[0] <= res.back()[1])
            res.back()[1] = max(res.back()[1], in[1]);
        else res.push_back(in);
    }
    return res;
}
```

PYTHON

```
def merge_intervals(iv):
    iv = sorted(iv); res = []
    for s, e in iv:
        if res and s <= res[-1][1]:
            res[-1][1] = max(res[-1][1], e)
        else: res.append([s, e])
    return res
```

Complexity & notes: $O(n \log n)$ (the sort dominates), $O(n)$ output. Sort-then-sweep is the template for all interval problems.

Q75. Merge Two Sorted Arrays (no extra space)

Hard

■ **Approach:** Gap method (Shell-sort idea). Treat the two arrays as one virtual array of length $n+m$. Start with $\text{gap} = \text{ceil}((n+m)/2)$; compare elements 'gap' apart and swap if out of order; halve the gap each round until it's 0. Achieves sorted merge in $O(1)$ extra space.

C++

```
// at(i)/setv(i) index into a (i<n) or b (i-n)
void mergeNoExtra(vector<int>& a, vector<int>& b){
    int n=a.size(), m=b.size();
    auto at=[&](int i)->int&{ return i<n ? a[i] : b[i-n]; };
    int gap=(n+m+1)/2;
    while(gap>0){
        int i=0, j=gap;
        while(j<n+m){
            if(at(i)>at(j)) swap(at(i),at(j));
            i++; j++;
        }
        gap = (gap==1) ? 0 : (gap+1)/2;
    }
}
```

PYTHON

```
def merge_no_extra(a, b):
    n, m = len(a), len(b)
    def at(i): return a[i] if i<n else b[i-n]
    def setv(i,v):
        if i<n: a[i]=v
        else: b[i-n]=v
    gap = (n+m+1)//2
    while gap>0:
        i, j = 0, gap
        while j<n+m:
            if at(i)>at(j):
                x,y=at(i),at(j); setv(i,y); setv(j,x)
                i+=1; j+=1
            gap = 0 if gap==1 else (gap+1)//2
        # a then b are now globally sorted
```

Complexity & notes: $O((n+m) \log(n+m))$ time, $O(1)$ space. Beats the simple $O(n+m)$ -space merge when in-place is required.

Q76. Find the Repeating and Missing Number

Hard

■ **Approach:** Numbers $1..n$ with one repeated (x) and one missing (y). From the sums: $(\text{sum} - \text{expected}) = x - y$, and $(\text{sumSquares} - \text{expectedSquares}) = x^2 - y^2 = (x-y)(x+y)$. Divide to get $x+y$, then solve the two linear equations for x and y .

C++

```
vector<int> repeatMissing(vector<int>& a){
    long n=a.size();
    long S=n*(n+1)/2, S2=n*(n+1)*(2*n+1)/6, su=0, su2=0;
    for(int x : a){ su+=x; su2+=(long)x*x; }
    long d=su-S;          // x - y
    long s=(su2-S2)/d;     // x + y
    long x=(d+s)/2, y=x-d;
    return {(int)x,(int)y}; // {repeating, missing}
}
```

PYTHON

```
def repeat_missing(a):
    n=len(a)
    S=n*(n+1)//2; S2=n*(n+1)*(2*n+1)//6
    su=sum(a); su2=sum(x*x for x in a)
    d=su-S; s=(su2-S2)//d      # x-y, x+y
    x=(d+s)//2; y=x-d
    return [x, y]             # [repeating, missing]
```

Complexity & notes: $O(n)$ time, $O(1)$ space. Use long/long long for the squared sums (they overflow int quickly). XOR method also works.

Q77. Count Inversions

Hard

■ **Approach:** An inversion is a pair $i < j$, $a[i] > a[j]$. Piggyback on MERGE SORT: while merging two sorted halves, if a left element is bigger than a right element, ALL remaining left elements are also bigger - add $(mid - i + 1)$ to the count. Sorting side-effect gives the count in $O(n \log n)$.

C++

```
long merge(vector<int>& a, int l, int r){
    if(l>=r) return 0;
    int m=(l+r)/2;
    long inv=merge(a,l,m)+merge(a,m+1,r);
    vector<int> t; int i=l,j=m+1;
    while(i<=m&& j<=r){
        if(a[i]<=a[j]) t.push_back(a[i++]);
        else { t.push_back(a[j++]); inv += m-i+1; } // key
    }
    while(i<=m) t.push_back(a[i++]);
    while(j<=r) t.push_back(a[j++]);
    for(int k=0;k<(int)t.size();k++) a[l+k]=t[k];
    return inv;
}
```

PYTHON

```
def count_inversions(a):
    def sort(arr):
        if len(arr)<=1: return arr, 0
        m=len(arr)//2
        L,cl=sort(arr[:m]); R,cr=sort(arr[m:])
        merged=[]; i=j=inv=0
        while i<len(L) and j<len(R):
            if L[i]<=R[j]: merged.append(L[i]); i+=1
            else: merged.append(R[j]); j+=1; inv+=len(L)-i
        return merged+L[i:]+R[j:], cl+cr+inv
    return sort(a)[1]
```

Complexity & notes: $O(n \log n)$ time. Brute force is $O(n^2)$. The 'add all remaining left elements' step is the whole trick.

Q78. Reverse Pairs

Hard

■ **Approach:** Count pairs $i \cdot 2 \cdot a[j]$. Merge-sort based like inversions, but the counting is a SEPARATE step before merging: for each left element, advance a pointer over the right half counting how many satisfy $a[i] > 2 \cdot a[j]$. Then merge normally.

C++

```
long merge(vector<int>& a, int l, int r){
    if(l>=r) return 0;
    int m=(l+r)/2;
    long cnt=merge(a,l,m)+merge(a,m+1,r);
    int j=m+1;
    for(int i=l;i<=m;i++){
        // count first
        while(j<=r && (long)a[i] > 2L*a[j]) j++;
        cnt += j-(m+1);
    }
    vector<int> t; int i=l; j=m+1;
    // then merge
    while(i<=m&&j<=r) t.push_back(a[i]<=a[j]?a[i++]:a[j++]);
    while(i<=m) t.push_back(a[i++]);
    while(j<=r) t.push_back(a[j++]);
    for(int k=0;k<(int)t.size();k++) a[l+k]=t[k];
    return cnt;
}
```

PYTHON

```
def reverse_pairs(a):
    def sort(arr):
        if len(arr)<=1: return arr, 0
        m=len(arr)//2
        L,cl=sort(arr[:m]); R,cr=sort(arr[m:])
        cnt=0; j=0
        for x in L:
            # count first
            while j<len(R) and x > 2*R[j]: j+=1
            cnt += j
        return sorted(L+R), cl+cr+cnt
    # then merge
    return sort(a)[1]
```

Complexity & notes: $O(n \log n)$. The counting pass is kept separate from the merge because the $2 \cdot a[j]$ condition doesn't align with the merge comparison.

Q79. Maximum Product Subarray

Medium

■ **Approach:** Track BOTH the max-product and min-product ending here - because a negative number flips a big negative (min) into a big positive (max). On a negative element, swap max and min first, then update each with x vs (running * x). Track the best max.

C++

```
int maxProduct(vector<int>& a){
    int best=a[0], mx=a[0], mn=a[0];
    for(int i=1;i<(int)a.size();i++){
        int x=a[i];
        if(x<0) swap(mx, mn);      // negatives flip roles
        mx = max(x, mx*x);
        mn = min(x, mn*x);
        best = max(best, mx);
    }
    return best;
}
```

PYTHON

```
def max_product(a):
    best = mx = mn = a[0]
    for x in a[1:]:
        if x < 0: mx, mn = mn, mx    # negatives flip roles
        mx = max(x, mx*x)
        mn = min(x, mn*x)
        best = max(best, mx)
    return best
```

Complexity & notes: $O(n)$, $O(1)$. Tracking the MIN too is the key - a negative times the min can become the new max.

Binary Search - Start (Q80)

Q80. Binary Search to Find X

Easy

■ **Approach:** On a SORTED array, keep two pointers lo, hi. Look at the middle; if it equals the target, done. If middle is too small, the answer (if any) is to the right, so move lo past mid. If too big, move hi before mid. Each step halves the search range.

C++

```
int binarySearch(vector<int>& a, int t){
    int lo=0, hi=a.size()-1;
    while(lo <= hi){
        int mid = lo + (hi-lo)/2;    // overflow-safe
        if(a[mid]==t) return mid;
        else if(a[mid]<t) lo = mid+1;
        else hi = mid-1;
    }
    return -1;
}
```

PYTHON

```
def binary_search(a, t):
    lo, hi = 0, len(a)-1
    while lo <= hi:
        mid = lo + (hi-lo)//2    # overflow-safe
        if a[mid]==t: return mid
        elif a[mid]<t: lo = mid+1
        else: hi = mid-1
    return -1
```

Complexity & notes: $O(\log n)$ time, $O(1)$ space. The skeleton for the entire Binary Search step - lo/hi/mid + 'discard half'. Requires sorted input.

That's Q61-80 - the hard array problems, done. The big new techniques: **merge-sort-based counting** (inversions, reverse pairs), **sort + two pointers** (3-Sum, 4-Sum), **prefix sum/XOR + hashmap** (subarrays with sum/XOR K, zero-sum), the **markers-in-place** trick (set matrix zeros), and matrix walks (spiral, rotate). Q80 kicks off Binary Search - the next 30+ problems all build on that lo/hi/mid skeleton. Re-type the hard ones (3-Sum, inversions, max product) from a blank page until they're automatic. Next: Q81-100 (the full Binary Search step).